

Dimensionality reduction

Mathematics of AI - RPTU Kaiserslautern

WiSe 25/26

Topics of the lecture

- Linear dimensionality reduction
 - Principal component analysis
- Nonlinear dimensionality reduction
 - tSNE, UMAP

PCA: refresher

Principal Component Analysis is a dimensionality reduction technique, solving the following optimization problem

$$[Q^*, \mu^*] = \operatorname{argmin}_{Q, \mu} \sum_{t=1}^T \|x_t - (QQ^T(x_t - \mu) + \mu)\|_2^2$$

PCA algorithm

PCA

- 1 **input:** data X , number of components m .
 - 2 Subtract mean from data (for each dimension)
 - 3 Construct covariance matrix
 - 4 Get the m largest eigenvalues, and corresponding eigenvectors
 - 5 Construct Q
 - 6 Project the data $X_{\text{reduced}} = Q^T X_{\text{mean removed}}$
-

Error

We can calculate

$$\text{err}_{\text{rel}} := \frac{\sum_{i=m+1}^n \lambda_i}{\sum_{i=1}^n \lambda_i}$$

and

$$\text{var}_{\text{expl}} = (1 - \text{err}_{\text{rel}}) \cdot 100$$

which is the information retained by the reduced representation of X .

PCA using `scikit-learn`

The `scikit-learn` library offers an implementation of PCA

```
from sklearn.decomposition import PCA

m = PCA(
    n_components = 2, # number of PCA components to keep
    ...
).fit(X)
```

For more options, see the documentation

Properties

`mod.components_` # PCs (sorted)

`mod.explained_variance_` # variance expl. by each component

`mod.explained_variance_ratio_`

`mod.mean_`

`mod.singular_values_`

tSNE

T-distributed Stochastic Neighbors Embedding (tSNE) can be used to visualize high-dimensional data. We can use the sklearn-provided class

```
from sklearn.manifold import TSNE
mod = TSNE(
    n_components,
    perplexity=30.0,
    learning_rate="auto" # or float [10.0, 1000.0]
    max_iter = 1000
).fit(X)
mod.embedding_ # representations vector
```


UMAP

Uniform Manifold Approximation & Projection (UMAP) is not included in sklearn, but in umap-learn. See [the docs](#)

```
from umap import UMAP
mod = UMAP(
    n_components = 2,
    n_neighbors=15, # controls structure of output
    min_dist=0.1, # minimal allowed distance in reduced space
    metric="euclidean", # metric to be used
).fit(X)
X_emb = mod.transform(X)
```

Additional dimensionality reduction methods

Within sklearn, there are additional variations that can be considered, for example:

- KernelPCA, using a kernel
- SparsePCA, l1 regularized PCA

Try the methods on different datasets

Explore the methods by applying them to some additional dataset.

- have a look at [the sklearn.dataset documentation](#)
- select some interesting dataset
- apply the methodologies
 - try changing hyperparameters
- observe the results

note: the datasets may be referring to a classification/regression problem, but in this case we are using unsupervised dimensionality reduction methods, so the labels can be ignored.